

Práctica: Autenticación de Usuario con Verificación por PIN (2FA)

Nombre: Daniel Galicia Cobaxin

Materia: Aplicaciones Distribuidas

Fecha: 10 de julio de 2026

Introducción

En esta práctica se implementó un sistema de acceso con dos pasos de verificación. Primero el usuario captura su usuario y contraseña, y si son correctos el sistema genera un Token y un PIN de 6 dígitos que se muestran en la terminal del servidor. Después, el usuario debe capturar ese PIN en una segunda pantalla para poder entrar finalmente a una página de bienvenida. Con esto se busca reforzar la seguridad del inicio de sesión, ya que aunque alguien conozca la contraseña de un usuario, no podrá entrar sin el PIN que solo aparece en el servidor.

El objetivo de este reporte es describir cómo se desarrolló la práctica, qué archivos se utilizaron y cómo se comprobó su funcionamiento por medio de distintos casos de uso, dejando espacio para las evidencias correspondientes.

Desarrollo

Archivos utilizados

script.js — Se encarga del formulario de login. Toma el usuario y la contraseña capturados, los envía al endpoint `/valida_login` y, según la respuesta, muestra un mensaje de error o guarda el usuario y el token en `localStorage` para pasar a la pantalla del PIN (`pin.html`).

pin.js — Recupera el usuario guardado en `localStorage` y lo muestra en el formulario del PIN. Envía el PIN capturado al endpoint `/valid_pin` y, si es válido, redirige a `bienvenido.html`; si no, muestra el mensaje de error que regresa el servidor.

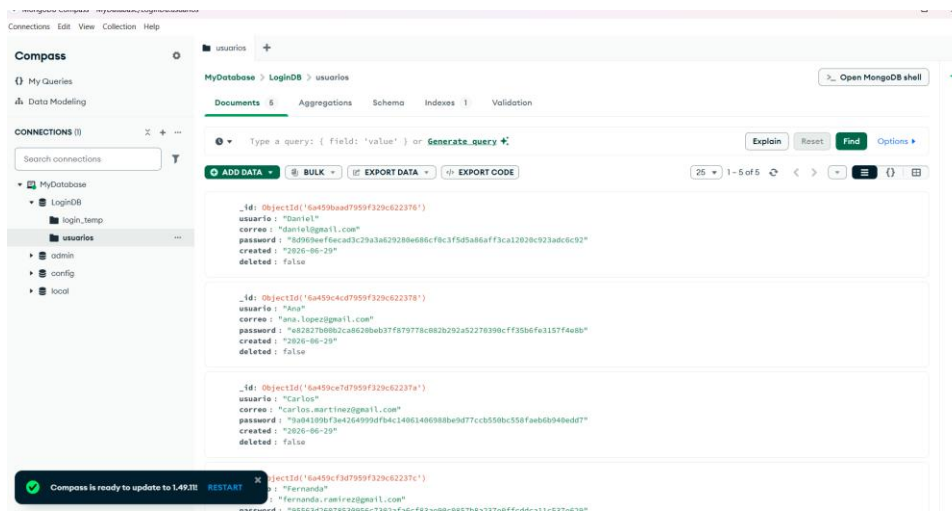
style.css — Da estilo a la pantalla de login: centra el formulario, le da fondo blanco, bordes redondeados y da formato a los campos de texto y botones.

usuarios.js — Es el archivo de rutas (router) de Express. Contiene el endpoint `/valida_login`, que valida el usuario y la contraseña contra la colección `usuarios` de MongoDB y genera el token y el PIN, y el endpoint `/valid_pin`, que valida el PIN contra la colección `login_temp`, revisa su expiración y lo elimina una vez usado.

usuariosController.js — Archivo reservado para separar la lógica de negocio del router en un futuro; en esta versión de la práctica se dejó vacío y toda la lógica se manejó directamente en `usuarios.js`.

Base de datos

Se utilizó MongoDB con la base de datos `LoginDB`, revisada desde MongoDB Compass. La colección `usuarios` guarda a los usuarios registrados con su usuario, correo y contraseña ya convertida a SHA256. La colección `login_temp` guarda de forma temporal el token, el PIN, la fecha de creación y la fecha de expiración de cada intento de login, y sus registros se eliminan automáticamente al validarse el PIN correctamente.



Colección usuarios en MongoDB Compass.

Flujo de la aplicación

El flujo general de la práctica queda de la siguiente manera: primero se captura el usuario y la contraseña, la contraseña se convierte a SHA256 y se compara contra lo guardado en MongoDB; si todo es correcto se genera un Token y un PIN; después el usuario debe validar ese PIN, y una vez validado se elimina el Token para que no se pueda volver a usar, entrando finalmente a la página de bienvenido.

Casos de uso

A continuación, se describen los casos de uso que se probaron, el objetivo de cada uno, el resultado que se esperaba y cómo se llevó a cabo usando los archivos ya descritos. Debajo de cada caso de uso se deja un espacio para colocar la evidencia correspondiente (captura de pantalla).

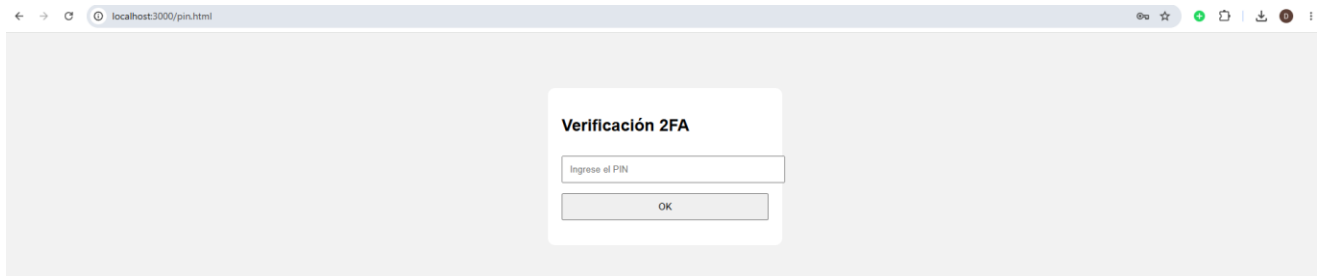
CU-01. Login exitoso

Objetivo: Verificar que un usuario registrado pueda iniciar sesión.

Resultado esperado: Se genera un Token y un PIN, y se muestra la pantalla para capturar el PIN.

Procedimiento: En login.html se capturan usuario y contraseña con script.js. Al enviar el formulario se llama la función login(), que hace un fetch hacia el endpoint /valida_login de usuarios.js. En el backend se busca el usuario en la colección usuarios de MongoDB y se compara la contraseña convertida a SHA256. Si coinciden, se genera un token con uuid y un PIN de 6 dígitos, y ambos se guardan en la colección login_temp con una expiración de 5 minutos. El PIN se imprime en la terminal del servidor con console.log. El backend responde con estado:0, por lo que script.js guarda el usuario y el token en localStorage y redirige a pin.html.



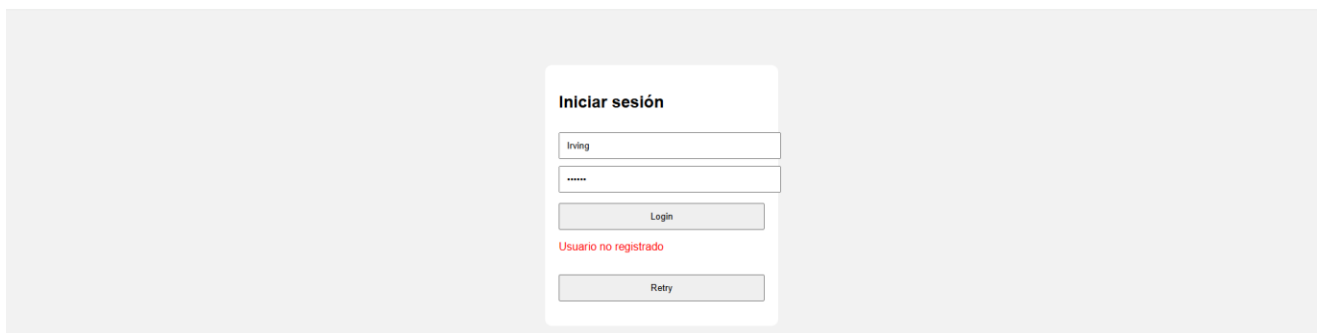


CU-02. Usuario inexistente

Objetivo: Verificar que no se permita iniciar sesión con un usuario que no existe.

Resultado esperado: Se muestra "Usuario no registrado" (estado = 1).

Procedimiento: Se escribe en el formulario un usuario que no existe en la colección usuarios. En usuarios.js, usuarios.findOne no encuentra ningún documento, por lo que el backend responde con estado:1 y el mensaje 'Usuario no registrado'. script.js recibe esa respuesta y la muestra en el div de respuesta junto con un botón para reintentar.

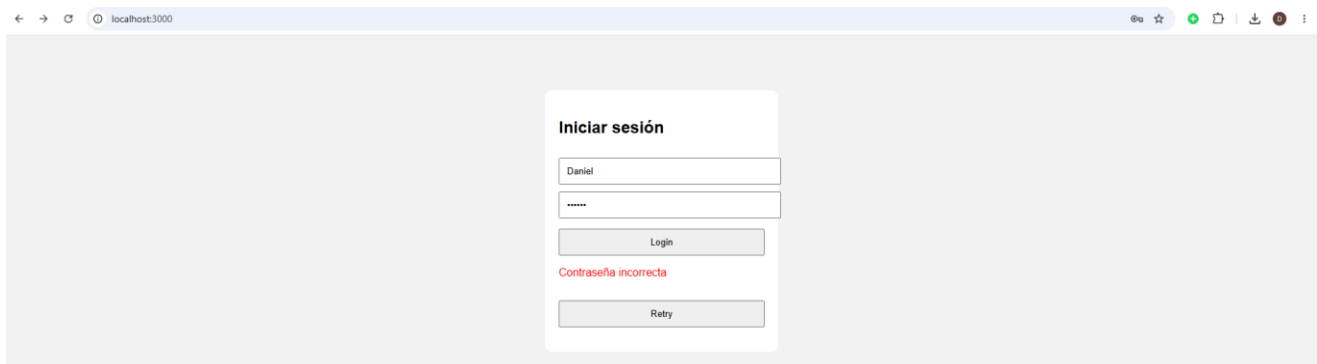


CU-03. Contraseña incorrecta

Objetivo: Verificar que no se permita iniciar sesión con una contraseña incorrecta.

Resultado esperado: Se muestra "Contraseña incorrecta" (estado = 2).

Procedimiento: Se escribe un usuario que sí existe, pero con una contraseña distinta a la registrada. El backend encuentra al usuario en usuarios.js, pero al comparar el hash SHA256 de la contraseña capturada contra el guardado en MongoDB, estos no coinciden. El backend responde con estado:2 y el mensaje 'Contraseña incorrecta', que script.js despliega en pantalla.

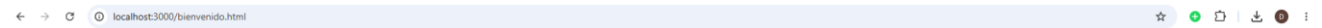
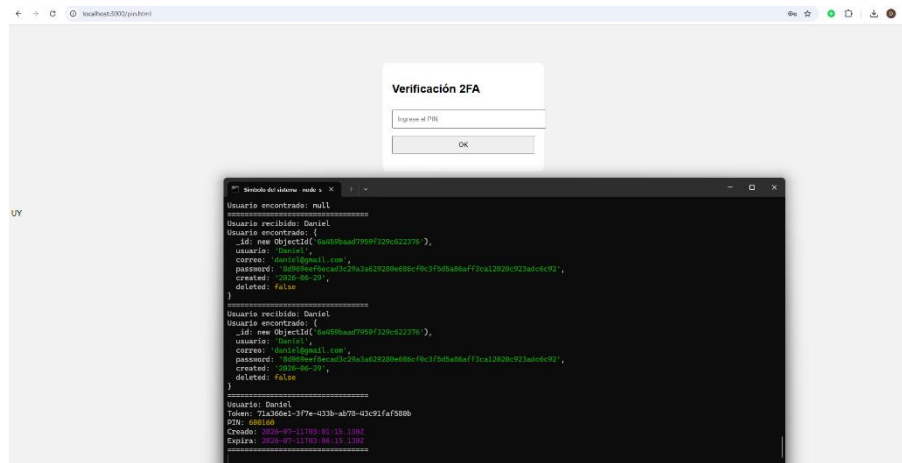


CU-04. PIN correcto

Objetivo: Verificar que el segundo factor de autenticación funcione correctamente.

Resultado esperado: Se muestra "OK - Access Successfully" y se redirige a bienvenido.html.

Procedimiento: Después de un login exitoso se llega a pin.html. Con pin.js se recupera el usuario guardado en localStorage y se pide capturar el PIN que se generó en la terminal. Al enviarlo se llama a validarPIN(), que hace un fetch hacia /valid_pin en usuarios.js. El backend busca el registro del usuario en login_temp, revisa que no haya expirado y compara el PIN capturado contra el guardado. Como coinciden, se elimina el registro de login_temp y se responde con valido:1, por lo que pin.js redirige a bienvenido.html.



Bienvenido

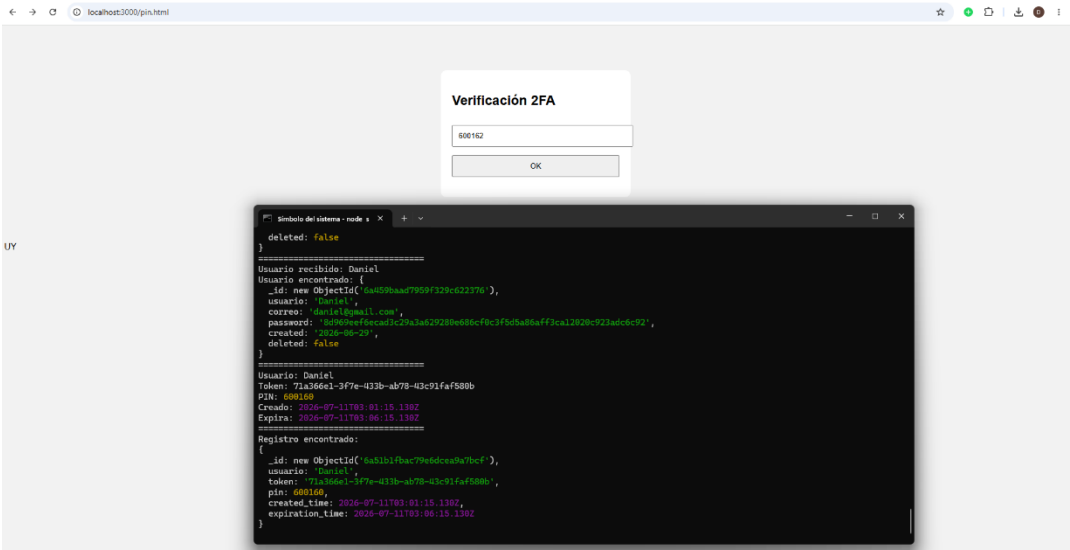
Autenticación en dos pasos correcta.

CU-05. PIN incorrecto

Objetivo: Verificar que un PIN diferente al generado sea rechazado.

Resultado esperado: Se muestra "PIN incorrecto".

Procedimiento: Se captura en pin.html un PIN distinto al que se generó en la terminal. En /valid_pin de usuarios.js, la comparación registro.pin != pin resulta verdadera, por lo que el backend responde con valido:0 y el mensaje 'PIN incorrecto.', que pin.js muestra en el div de respuesta.

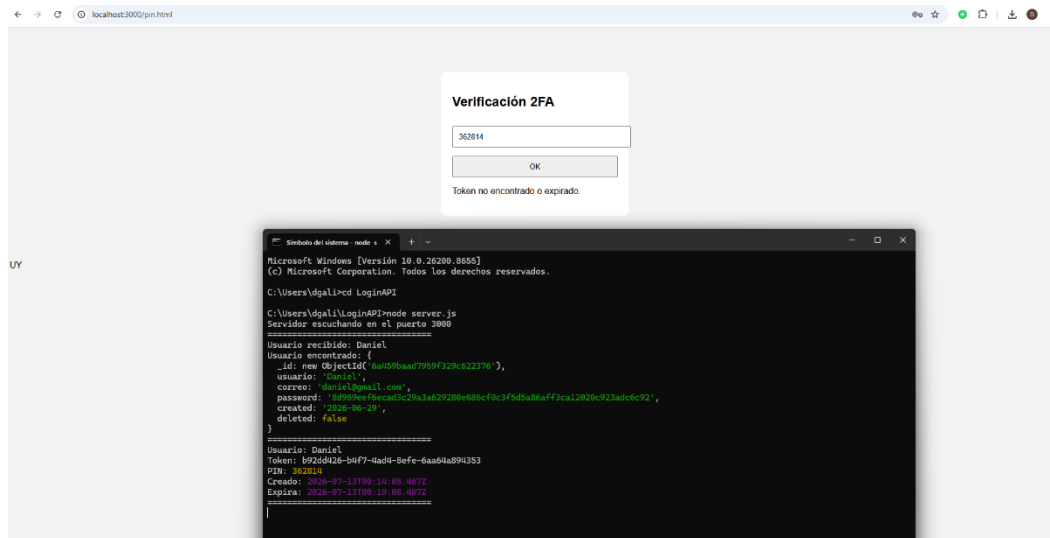
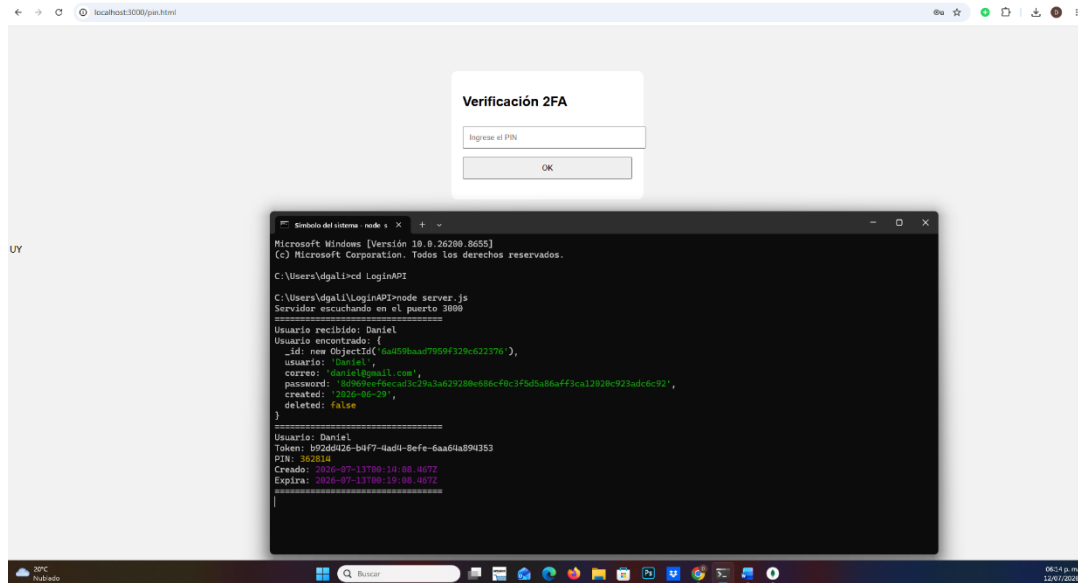


CU-06. PIN expirado

Objetivo: Verificar que un PIN ya vencido no pueda utilizarse.

Resultado esperado: Se muestra "El token ha expirado."

Procedimiento: Se deja pasar más de 5 minutos (tiempo definido en `expiration_time` al crear el registro) antes de capturar el PIN en `pin.html`. En `/valid_pin`, la condición ahora `> registro.expiration_time` se cumple, así que el backend responde con el mensaje 'El token ha expirado.' sin permitir el acceso, aunque el PIN capturado sea el correcto.



CU-07. Token eliminado

Objetivo: Verificar que el PIN no pueda reutilizarse después de un acceso exitoso.

Resultado esperado: El sistema rechaza un segundo intento porque el registro ya no existe en login_temp.

Procedimiento: Después de completar exitosamente el CU-04, se intenta capturar de nuevo el mismo PIN en pin.html. Como en /valid_pin el registro correspondiente ya fue eliminado de login_temp con deleteOne al validarse la primera vez, la búsqueda findOne no encuentra nada y el backend responde con el mensaje 'Token no encontrado o expirado.'



Conclusión

Con esta práctica se logró implementar un sistema de acceso más seguro, ya que no basta con conocer la contraseña de un usuario para entrar, también se necesita el PIN que genera el servidor y que expira en pocos minutos. Se reforzó el uso de Express para crear rutas, MongoDB para guardar tanto a los usuarios como los registros temporales de login, y SHA256 para no guardar las contraseñas en texto plano. También se comprobó, a través de los distintos casos de uso, que el sistema responde correctamente tanto cuando todo es correcto como cuando hay errores del usuario, contraseña, PIN incorrecto, PIN expirado o PIN reutilizado.